

NeetCode 150

Solutions & Commentary

Andre Ruiz Loera

Preface

I am a Mathematics major at the University of Illinois Urbana-Champaign and I have always enjoyed solving puzzles. Math gives you the peaceful kind of puzzle where you sit with an idea long enough and eventually something clicks. It is slow and thoughtful and satisfying when the pieces finally fall into place.

Coding feels different. It is still a puzzle but it is a louder one. And writing code in C++ feels like trying to solve a puzzle while you are on fire. It can be chaotic and confusing and still incredibly rewarding when it works. For a long time I actually became pretty good at the classic LeetCode style of problems. I had a sense for patterns and techniques and I could look at a question and feel the solution forming before I even finished the prompt.

Then I stopped for a while.

Life shifted toward other things and I did not work on these kinds of problems for months. When I returned I realized that my old LeetCode instincts had mostly evaporated. The skills were not gone forever but they were definitely sleeping. I felt like I had to start fresh again.

This book is my return to that world. It is my way of rebuilding my problem solving instincts one step at a time. The NeetCode 150 list is one of the clearest and most thoughtful collections of interview problems I know. Every problem teaches something specific and every solution strengthens a part of your thinking.

My goal in writing this is simple. I want to explain how I approach each problem. I want to write out the thought process the honest confusion the small breakthroughs and the cleanest version of a C++ solution that I can manage. I want this book to feel like you are sitting next to someone who is also working through these problems and talking through each idea as it forms.

If you are reading this I hope you find something helpful. Maybe a new way to look at a problem or the reassurance that you are not the only one who gets stuck

sometimes. This is my journey back to algorithmic thinking and I am treating every problem as a chance to get sharper again.

Welcome to the process. And good luck. May your solutions arrive faster than they did for me when I first picked this back up.

What is NeetCode 150 and Why Use It

The NeetCode 150 list is a curated set of one hundred and fifty coding interview problems gathered from the most common patterns that appear in technical interviews. It was created by NeetCode, a well-respected educator whose videos and write-ups focus on building real understanding rather than just memorizing tricks.

The list is organized by topic and by problem pattern. Each problem represents a skill that shows up repeatedly in interviews. For example sliding window, binary search, graph traversal, dynamic programming and many others. Because the list is structured and selective it gives you a very efficient way to study. You spend time on the problems that matter the most instead of trying to solve thousands of random questions with unclear purpose.

People use the NeetCode 150 to prepare for coding interviews at all levels. It covers the core ideas that interviewers expect candidates to know. It builds intuition about data structures and algorithms and it makes you better at recognizing patterns and adapting them to new problems.

I am using it because it is one of the most focused and effective paths back into algorithmic problem solving. It gives me a defined starting point and a clear progression. Working through the list forces me to practice thinking clearly under constraints and it gives me a chance to rebuild confidence and speed in a structured way.

In short the NeetCode 150 is a well-designed roadmap. It teaches the fundamentals that matter and prepares you for the type of reasoning that technical interviews and real-world problem solving require.

The Learning Roadmap

The NeetCode 150 follows a structured progression that builds fundamental skills before advancing to more complex patterns. The roadmap below illustrates how topics connect and build upon each other:

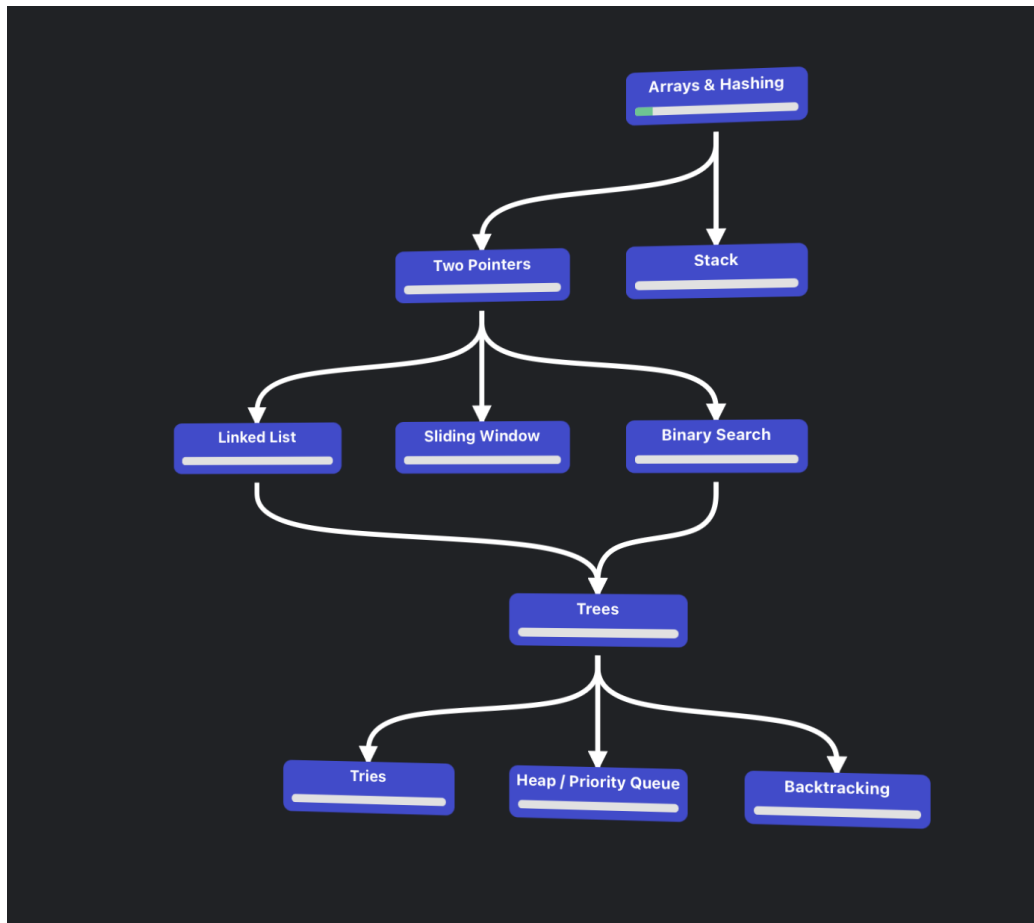


Figure 1: NeetCode 150 Learning Roadmap - Topic Progression

This visual guide shows the recommended path through the problems. Starting with Arrays & Hashing establishes the foundation, then branching into Two Pointers and Stack techniques. From there, the path expands to cover Linked Lists, Sliding Window, and Binary Search. These foundational topics converge into Trees, which then branches into more specialized areas like Tries, Heaps, and Backtracking.

Contents

Preface	3
1 Arrays & Hashing	11
1.1 Contains Duplicate	11
1.1.1 First Attempt and Why It Was Incorrect	11
1.2 Top K Frequent Elements	12
1.3 Product of Array Except Self	12
1.4 Valid Sudoku	12
1.5 Encode and Decode Strings	12
1.6 Longest Consecutive Sequence	12
2 Two Pointers	13
2.1 Valid Palindrome	13
2.2 Two Sum II	13
2.3 3Sum	13
2.4 Container With Most Water	13
2.5 Trapping Rain Water	13
3 Sliding Window	15
3.1 Best Time to Buy and Sell Stock	15
3.2 Longest Substring Without Repeating Characters	15
3.3 Longest Repeating Character Replacement	15
3.4 Minimum Window Substring	15
4 Stack	17
4.1 Valid Parentheses	17
4.2 Min Stack	17

4.3	Evaluate Reverse Polish Notation	17
4.4	Generate Parentheses	17
4.5	Daily Temperatures	17
4.6	Car Fleet	17
4.7	Largest Rectangle in Histogram	17
5	Binary Search	19
5.1	Binary Search	19
5.2	Search a 2D Matrix	19
5.3	Koko Eating Bananas	19
5.4	Find Minimum in Rotated Sorted Array	19
5.5	Search in Rotated Sorted Array	19
5.6	Time Based Key-Value Store	19
5.7	Median of Two Sorted Arrays	19
6	Linked List	21
6.1	Reverse Linked List	22
6.2	Merge Two Sorted Lists	22
6.3	Reorder List	22
6.4	Remove Nth Node From End of List	22
6.5	Copy List with Random Pointer	22
6.6	Add Two Numbers	22
6.7	Linked List Cycle	22
6.8	Find the Duplicate Number	22
6.9	LRU Cache	22
6.10	Merge K Sorted Lists	22
6.11	Reverse Nodes in K-Group	22
7	Trees	23
7.1	Invert Binary Tree	24
7.2	Maximum Depth of Binary Tree	24
7.3	Diameter of Binary Tree	24
7.4	Balanced Binary Tree	24
7.5	Same Tree	24
7.6	Subtree of Another Tree	24

7.7	Lowest Common Ancestor of a Binary Search Tree	24
7.8	Binary Tree Level Order Traversal	24
7.9	Binary Tree Right Side View	24
7.10	Count Good Nodes in Binary Tree	24
7.11	Validate Binary Search Tree	24
7.12	Kth Smallest Element in a BST	24
7.13	Construct Binary Tree from Preorder and Inorder Traversal	24
7.14	Binary Tree Maximum Path Sum	24
7.15	Serialize and Deserialize Binary Tree	24
8	Tries	25
8.1	Implement Trie	25
8.2	Design Add and Search Words Data Structure	25
8.3	Word Search II	25
9	Heap / Priority Queue	27
9.1	Kth Largest Element in a Stream	27
9.2	Last Stone Weight	27
9.3	K Closest Points to Origin	27
9.4	Kth Largest Element in an Array	27
9.5	Task Scheduler	27
9.6	Design Twitter	27
9.7	Find Median from Data Stream	27
10	Backtracking	29
10.1	Subsets	30
10.2	Combination Sum	30
10.3	Permutations	30
10.4	Subsets II	30
10.5	Combination Sum II	30
10.6	Word Search	30
10.7	Palindrome Partitioning	30
10.8	Letter Combinations of a Phone Number	30
10.9	N-Queens	30

Chapter 1

Arrays & Hashing

1.1 Contains Duplicate

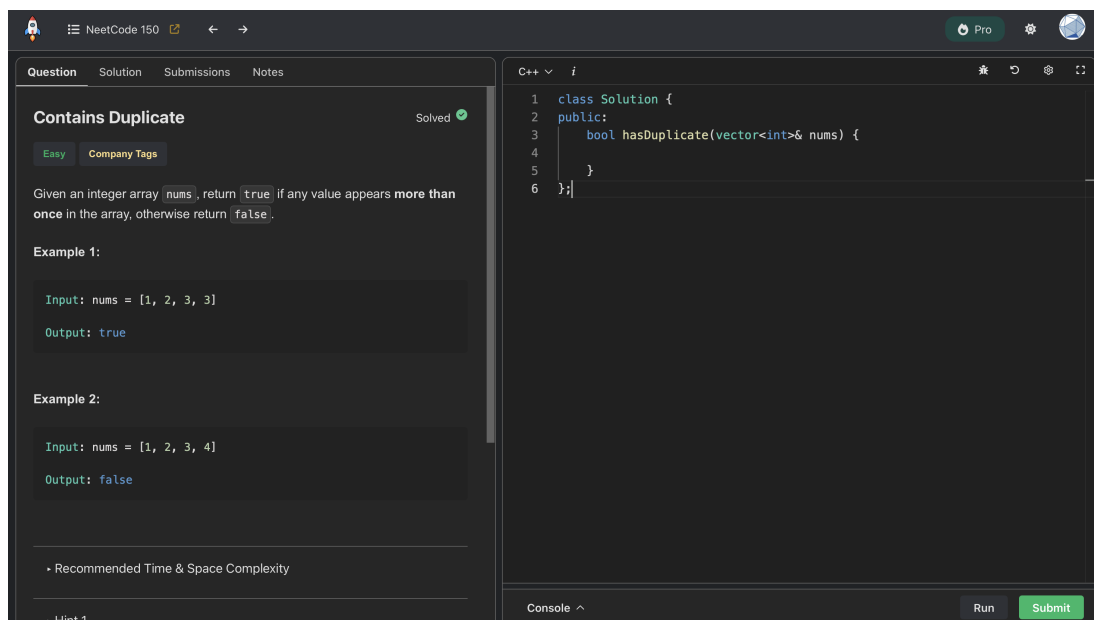


Figure 1.1: Problem 1: Contains Duplicate - Given an array of integers, determine if any value appears at least twice

1.1.1 First Attempt and Why It Was Incorrect

In my initial attempt, I began by examining the function signature:

```
1 bool hasDuplicate(vector<int>& nums)
```

I correctly understood that the function returns a boolean value, meaning it must return either `true` or `false` depending on whether a duplicate exists in the input. I also understood that `vector<int>` represents a dynamic array of integers, and that the parameter name `nums` is simply what we call this array inside the function. Additionally, I did have a correct conceptual understanding of references: the ampersand (`&`) indicates that the vector is passed by reference, allowing the function to access the original data directly without copying it, which improves efficiency.

The issue in my first attempt was not with understanding the function signature but with the approach I considered for solving the problem. I initially thought of comparing each integer in the vector with every other integer. Although this brute-force approach would eventually detect a duplicate, it requires nested loops and runs in $O(n^2)$ time. This does not meet the problem's requirement of achieving $O(n)$ time complexity. I recognized that this naive method was inefficient but had not yet identified the appropriate data structure or strategy needed to reach the desired complexity.

Thus, the main flaw in my first attempt was the time complexity of the solution method, not a misunderstanding of the function's return type or the use of references.

1.2 Top K Frequent Elements

1.3 Product of Array Except Self

1.4 Valid Sudoku

1.5 Encode and Decode Strings

1.6 Longest Consecutive Sequence

Chapter 2

Two Pointers

2.1 Valid Palindrome

2.2 Two Sum II

2.3 3Sum

2.4 Container With Most Water

2.5 Trapping Rain Water

Chapter 3

Sliding Window

3.1 Best Time to Buy and Sell Stock

3.2 Longest Substring Without Repeating
Characters

3.3 Longest Repeating Character Replacement

3.4 Minimum Window Substring

Chapter 4

Stack

4.1 Valid Parentheses

4.2 Min Stack

4.3 Evaluate Reverse Polish Notation

4.4 Generate Parentheses

4.5 Daily Temperatures

4.6 Car Fleet

4.7 Largest Rectangle in Histogram

Chapter 5

Binary Search

5.1 Binary Search

5.2 Search a 2D Matrix

5.3 Koko Eating Bananas

5.4 Find Minimum in Rotated Sorted Array

5.5 Search in Rotated Sorted Array

5.6 Time Based Key-Value Store

5.7 Median of Two Sorted Arrays

Chapter 6

Linked List

6.1 Reverse Linked List

6.2 Merge Two Sorted Lists

6.3 Reorder List

6.4 Remove Nth Node From End of List

6.5 Copy List with Random Pointer

6.6 Add Two Numbers

6.7 Linked List Cycle

6.8 Find the Duplicate Number

6.9 LRU Cache

6.10 Merge K Sorted Lists

6.11 Reverse Nodes in K-Group

Chapter 7

Trees

7.1 Invert Binary Tree

7.2 Maximum Depth of Binary Tree

7.3 Diameter of Binary Tree

7.4 Balanced Binary Tree

7.5 Same Tree

7.6 Subtree of Another Tree

7.7 Lowest Common Ancestor of a Binary Search Tree

7.8 Binary Tree Level Order Traversal

7.9 Binary Tree Right Side View

7.10 Count Good Nodes in Binary Tree

Chapter 8

Tries

8.1 Implement Trie

8.2 Design Add and Search Words Data Structure

8.3 Word Search II

Chapter 9

Heap / Priority Queue

9.1 Kth Largest Element in a Stream

9.2 Last Stone Weight

9.3 K Closest Points to Origin

9.4 Kth Largest Element in an Array

9.5 Task Scheduler

9.6 Design Twitter

9.7 Find Median from Data Stream

Chapter 10

Backtracking

10.1 Subsets

10.2 Combination Sum

10.3 Permutations

10.4 Subsets II

10.5 Combination Sum II

10.6 Word Search

10.7 Palindrome Partitioning

10.8 Letter Combinations of a Phone Number

10.9 N-Queens